UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

ANÁLISIS DE SENTIMIENTO EN RESEÑAS DE HOTELES MEDIANTE APRENDIZAJE AUTOMÁTICO

PARCIAL 1

Integrantes:

- Sergio David Cambara Dorbigny 217163319
- Elizabeth Peña Rivero 215053745
- Claudia Evelin Tapia Molina 219072566

Materia: Tópicos Avanzados de Programación
Docente: Ing. Kenji Kawaida

Gestión 2-2026
Santa Cruz- Bolivia

ÍNDICE

ETAPA 1. INVESTIGACIÓN Y SELECCIÓN DE LOS DATOS....3
1.1 Investigación de conjunto de datos.... 3
1.2 Origen y especificaciones del dataset.... 3
1.3 Conversión de XML a tabla.... 4
1.4 Creación de la etiqueta sentimiento.... 4
1.5 Revisión manual de la calidad de etiquetado.... 4
1.6 Análisis exploratorio de los datos (EDA).... 5
1.7 Pipeline de preprocesamiento.... 9
ETAPA 2. MODELADO.... 11
2.1 Selección de los modelos.... 11
2.2 Configuración del experimento.... 12
2.3 Línea base.... 12
2.4 Ajuste de hiperparámetros....13
2.5 Comparación de los modelos.... 13
2.6 Modelo elegido para producción....14
2.7 Evaluación final en el conjunto de prueba.... 14
2.8 Matriz de confusión....15
2.9 Análisis de errores.... 16
2.10 Limitaciones y posibles mejoras.... 17
2.11 Artefactos y reproducibilidad....17
2.12 Declaración de uso de asistentes de IA.... 18
ETAPA 3. PREPROCESAMIENTO FINAL Y MODELO EN PRODUCCIÓN....18
3.1 Motivación....18
3.2 Decisiones de preprocesamiento (versión final)....18
3.3 Reentrenamiento y comparación de modelos.... 19
3.4 Evaluación final en el conjunto de prueba.... 19
3.5 Efecto del stemming.... 20
3.6 Artefactos y reproducibilidad.... 20
ETAPA 4. APLICACIÓN WEB: BACKEND, FRONTEND Y DESPLIEGUE.... 21
4.1 Arquitectura.... 21
4.2 Backend (FastAPI)....21
4.3 Frontend (React y Tailwind CSS)....22
4.4 Pruebas automatizadas....22
4.5 Ejecución y contenedores....23

ETAPA 1. INVESTIGACIÓN Y SELECCIÓN DE LOS DATOS

1.1 Investigación de conjunto de datos

Se realizo la búsqueda de conjuntos de datos relacionados con reseñas de hoteles, con el propósito de encontrar información adecuada para desarrollar un sistema de análisis de sentimiento mediante técnicas de procesamiento de lenguaje natural (PLN) y aprendizaje automático.

La búsqueda se centró principalmente en la plataforma Hugging Face, donde se identificaron diferentes conjuntos de datos con opiniones de clientes sobre hoteles. Se tomaron criterios primordiales como el idioma de la reseña, la cantidad de registros disponibles, la presencia de etiquetas de sentimientos, la calidad de los textos como ser valores faltantes, reseñas vacías, identificadores vacíos, textos duplicados, errores de formato, etiquetas inconsistentes, la facilidad de descarga y las condiciones de uso de los datos.

Entre las alternativas revisadas se encontraron conjuntos de datos en español e inglés. Sin embargo, algunas opciones presentaron dificultades relacionadas con el idioma, la organización de las columnas, la disponibilidad de los registros o la preparación necesaria para utilizarlos en un modelo.

1.2 Origen y especificaciones del dataset

El presente proyecto utiliza el corpus de reseñas COAH, creado por el grupo de investigación SINAI de la Universidad de Jaén, que trabaja en procesamiento del lenguaje natural (PLN).

Sus características principales son:

- **Idioma:** español
- **Tamaño:** 1,816 registros etiquetados
- **Dominio:** Reseñas de hoteles, estructuradas con id, puntuación de 1 a 5 (rank), abstract y review. Archivo original *corpus_coah.xml*
- **Licencia:** CC BY-NC-SA- 4.0 con uso académico y de investigación.
- **Fecha de obtención:** 21 de octubre de 2026
- **Link:** https://huggingface.co/datasets/SINAI/COAH/tree/main

1.3 Conversión de XML a tabla

Archivo: cargar_coah.py

El dataset original está en formato XML, que no se puede usar directamente para entrenar un modelo. El script cargar_coah.py lee cada reseña (hotel_review), extrae id, rank, abstract (título) y review (texto), y genera la tabla data/coah.csv con las columnas id, rank, titulo y texto.

1.4 Creación de la etiqueta sentimiento

Archivo: cargar_coah.py

El dataset no trae etiquetas de sentimiento, solo puntuaciones. Se derivo la columna sentimiento con estas reglas:1 y 2 estrellas, negativo; 3, neutro; 4 y 5, positivo.

El 3 se toma como neutro porque no expresa un rechazo ni una aprobación claros.

Esta regla es una decisión de diseño del proyecto, ya que ni el dataset ni la consigna fijan las clases. Se tomó el 3 como neutro porque no expresa un rechazo ni una aprobación claros, y se optó por tres clases para conservar todas las reseñas. Como limitación, una reseña de 3 estrellas puede mezclar aspectos buenos y malos, por lo que la clase neutra es la más difícil de clasificar.

1.5 Revisión manual de la calidad de etiquetado

Antes de realizar el análisis y seleccionar la muestra, se procesó el dataset original, compuesto por 1.816 registros, mediante un script en python para identificar y eliminar automáticamente las reseñas duplicadas. Este procedimiento permitió asegurar que cada reseña fuera única, reducir posibles sesgos por repetición de datos y verificar la calidad del conjunto de datos antes de entrenar los modelos de Machine Learning.

Posteriormente se realizó un muestreo aleatorio estratificado de 60 registros (20 positivos, 20 neutro y 20 negativos). La muestra se revisó manualmente para comprobar la correspondencia entre el contenido de las reseñas y sus etiquetas de sentimiento, con el fin de evaluar la calidad del etiquetado y la viabilidad del dataset para el entrenamiento de los modelos de Machine Learning.

Etiqueta	Revisadas	Sí	No	Dudosa	% coincide (Sí)
negativo	20	19	0	1	95%
neutro	20	16	3	1	80%
positivo	20	19	0	1	95%
Total	**60**	**54**	**3**	**3**	**90%**

Resultados

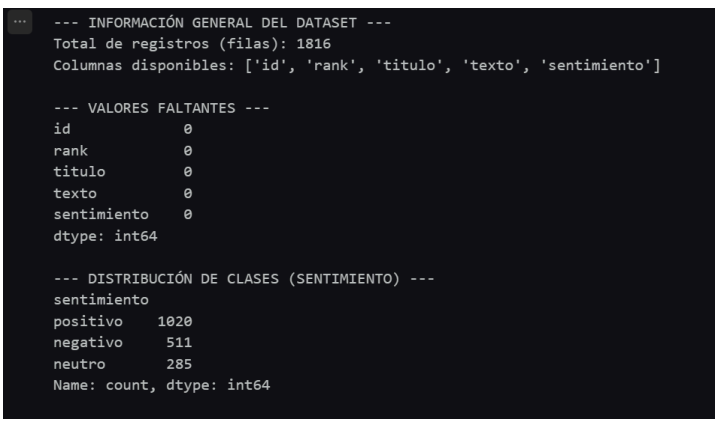
1. Reseñas positivas y negativas (95 % de coincidencia): La mayoría de estas reseñas tienen etiquetas correctas, indicando que las opiniones más claras se identifican con facilidad.

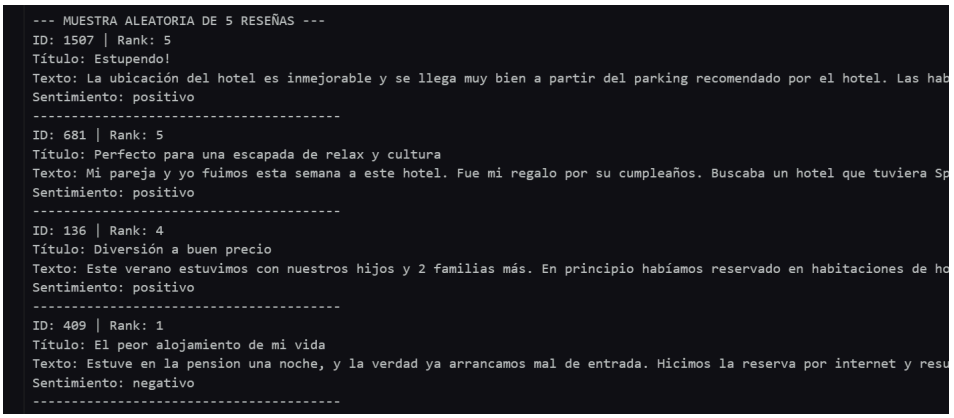
2. Reseñas neutras (80 % de coincidencia): Presentan un nivel aceptable, algunas opiniones tibias o mixtas que suelen ser más difíciles de clasificar.

3. Resultado general (90 % de coincidencia): En total, 54 de las 60 reseñas revisadas coincidieron con su etiqueta.

1.6 Análisis exploratorio de los datos (EDA)

En el análisis exploratorio de datos (EDA), se revisó y analizó el conjunto de datos para conocer su estructura y las características de las reseñas. Se identificaron un total de 1.816 registros y cinco columnas: id, rank, titulo, texto y sentimiento. También se verificó que no existen valores faltantes en ninguna de las columnas y se analizó la distribución de los sentimientos, obteniendo 1.020 reseñas positivas, 511 negativas y 285 neutras donde predominan las reseñas positivas, que representan aproximadamente el 56,2 % del conjunto de datos. Las negativas representan el 28,1 % y las neutras el 15,7 %.





Además, se revisó una muestra aleatoria de cinco reseñas para observar su contenido y comprobar la información registrada en cada una.

Duplicados

```
Textos duplicados: 24
Filas implicadas: 48
Pares con puntuación distinta: 0
```

Se detectaron 24 textos duplicados (48 filas), cada uno repetido una vez y con la misma puntuación en ambas copias, quedando 1.792 reseñas.

Longitud de los textos

```
count    1816.0
mean      128.3
std       105.2
min         7.0
25%        63.0
50%       101.0
75%       156.0
max      1416.0
Name: palabras, dtype: float64
sentimiento
negativo    166.7
neutro      132.6
positivo    107.9
Name: palabras, dtype: float64
Textos con menos de 20 palabras: 20
```

Las reseñas tienen en promedio 128 palabras (mediana 101; mínimo 7; máximo 1.416). Las negativas son más largas (167 palabras en promedio) que las neutras (133) y las positivas (108). Solo 20 reseñas tienen menos de 20 palabras.

Palabras más frecuentes

Negativo

[('hotel', 826), ('todo', 379), ('habitación', 326), ('nada', 304), ('este', 301), ('era', 278), ('habitaciones', 272), ('sin', 272), ('hay', 247), ('porque', 242)]

Neutro

[('hotel', 464), ('bien', 243), ('habitación', 187), ('hay', 174), ('todo', 169), ('está', 166), ('habitaciones', 146), ('poco', 133), ('esta', 128), ('desayuno', 118)]

Positivo

[('hotel', 1595), ('todo', 732), ('bien', 595), ('personal', 503), ('habitaciones', 479), ('este', 470), ('habitación', 412), ('desayuno', 350), ('sin', 350), ('está', 346)]

División del conjunto de datos

Antes de dividir, se eliminaron los 24 textos duplicados (de 1.816 a 1.792 reseñas) para evitar que una misma reseña quede en entrenamiento y en prueba.

- **Conjunto de entrenamiento (80%-1,433 registros)**
  Se utiliza para que el modelo aprenda a reconocer patrones en los textos y a relacionarlos con los sentimientos positivo, neutro y negativos.
- **Conjunto de prueba (20%-359 registros)**
  Se reserva para comprobar si el modelo puede clasificar correctamente reseñas que no utilizó durante el entrenamiento.

- **División estratificada y semilla fija**

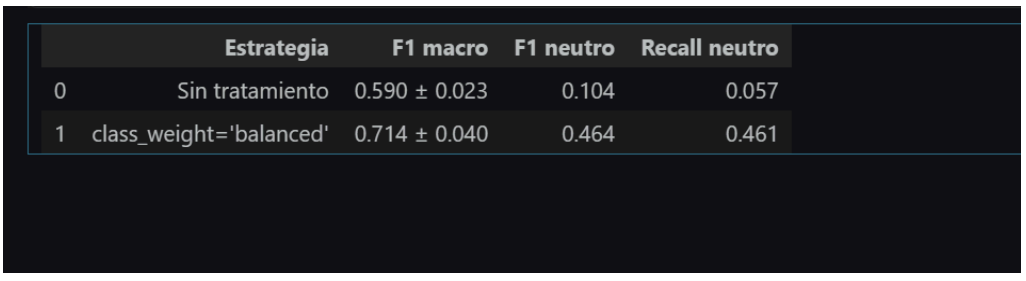
  Se utilizó stratify para mantener una proporción similar de reseñas positivas, neutras y negativas en ambos grupos. También se estableció random_state= 42 para que la división de los datos se repita de la misma manera cada vez que se ejecute el código con los mismos datos y parámetros.

- **Tratamiento justificado del desbalance de clases**

  Durante el análisis exploratorio de los datos del archivo coah.csv, se observó que había más opiniones positivas y negativas que neutras. Esta diferencia puede dificultar que el modelo identifique correctamente las opiniones neutras.

  Para abordar este problema, se utilizó la opción class_weight='balanced' en el modelo de Regresión Logística. Esta opción permite dar mayor importancia a las clases que tienen menos ejemplos, sin necesidad de agregar o eliminar reseñas del conjunto de datos.

  Posteriormente, se comparó el modelo con este ajuste y el modelo sin ajuste mediante la validación cruzada estratificada (StratifiedKFold). Para evaluar los resultados, se utilizaron las métricas F1-neutro y Recall-neutro, con el objetivo de comprobar si el ajuste mejoraba la identificación de las opiniones neutras.



	Estrategia	F1 macro	F1 neutro	Recall neutro
0	Sin tratamiento	0.590 ± 0.023	0.104	0.057
1	class_weight='balanced'	0.714 ± 0.040	0.464	0.461

  Los resultados de la validación cruzada mostraron que el modelo sin ajuste tenía dificultades para identificar las opiniones neutras, ya que su *Recall neutro* era de **0,057 (5,7 %)**. Esto significa que reconocía correctamente muy pocas de las reseñas que realmente eran neutras.

En cambio, al utilizar la opción class_weight='balanced', los resultados mejoraron. El **F1-macro aumentó de 0,590 (59,0 %) a 0,714 (71,4 %)**. Además, el modelo con ajuste obtuvo un **F1-neutro de 0,464 (46,4 %)** y un **Recall-neutro de 0,461 (46,1 %)**.

Estos resultados indican que el ajuste ayudó al modelo a identificar mejor las opiniones neutras y mejoró su rendimiento general. Por ello, se utilizó para tratar el desbalance de clases en el proyecto.

1.7 Pipeline de preprocesamiento

Demostración técnica de preprocesamiento

Archivo: demostracion_tecnicas_nlp.py

Para transformar el texto no estructurado en una representación apta para modelos de aprendizaje automático, se aplicaron y evaluaron las siguientes técnicas de Procesamiento de Lenguaje Natural (PLN):

1. **Normalización:** Conversión del texto a minúsculas, eliminación de signos de puntuación repetidos (ej. ¡¡...!!!), enlaces URL (https://...) y números.
2. **Filtrado de Palabras Vacías (*Stopwords*) y Conservación de Negaciones:** Se eliminaron palabras vacías estándar, pero se configuró una regla para conservar explícitamente la palabra "no" y términos de negación, dado su rol crítico en la identificación de reseñas negativas en el análisis de sentimientos.
3. Reducción Léxica (*Stemming* vs. Lematización):
    - **Stemming (Snowball):** Recorta las palabras a su raíz morfológica (ej. *habitación* → habit, *estaba* → estab)**.**
    - **Lematización (spaCy es_core_news_sm):** Reduce las palabras a su forma de lema válida en el diccionario (ej. *estaba* → estar).

Comparación de Experimentos y Representación en Vector

Archivo: experimentos_nlp.py

Se evaluaron 6 configuraciones experimentales combinando el método de reducción de palabras (ninguno, stemming, lematización) con el rango de N-gramas (unigramas frente a unigramas + bigramas) utilizando la representación TF-IDF.

Los resultados obtenidos en el conjunto de prueba (*Test*) se resumen a continuación:

Método de Reducción	**Representación (N-gramas)**	**F1 Macro (CV)**	**F1 Macro (Test)**	**Accuracy (Test)**	**Vocabulario**
Ninguno	Unigramas	0.7154	0.7076	78.27%	5,839
Ninguno	Unigramas + Bigramas	0.7161	0.6859	77.72%	15,485
Stemming	Unigramas	0.7147	**0.7202**	79.67%	**3,634**
Stemming	Unigramas + Bigramas	0.7199	0.7091	**81.06%**	16,099
Lematización	Unigramas	0.7075	0.7136	79.94%	4,462
Lematización	Unigramas + Bigramas	0.7094	**0.7203**	**81.06%**	16,608

Interpretación de los Resultados

1. Modelo con mejores resultados

La combinación de lematización + unigramas + bigramas fue la que obtuvo los mejores resultados al analizar las opiniones.

- F1 Macro de 0.7203: obtuvo una puntuación de 72,03% al evaluar qué tan bien identifica las diferentes clases de sentimientos.
- Accuracy de 81,06%: clasificó correctamente aproximadamente 81 de cada 100 opiniones.

2. Modelo que utiliza menos recursos

La combinación de stemming + unigramas obtuvo resultados prácticamente iguales al modelo anterior, con un F1 Macro de 0.7202 (72,02%).

Su ventaja es que utiliza un vocabulario de solo 3.634 palabras, lo que permite que el modelo sea más ligero y utilice menos memoria.

3. Efecto de las técnicas utilizadas

- **Reducción de palabras:** el stemming y la lematización ayudaron a mejorar los resultados. Estas técnicas permiten tratar palabras con diferentes formas como palabras relacionadas, lo que facilita el aprendizaje del modelo.
- **Uso de bigramas:** los bigramas permiten analizar pares de palabras juntas. Por ejemplo, no es lo mismo encontrar la palabra «recomiendo» que encontrar «no recomiendo». Esto ayudó al modelo a clasificar mejor algunas opiniones.

ETAPA 2. MODELADO

2.1 Selección de los modelos

El problema es una clasificación de texto en tres clases (negativo, neutro y positivo). Se eligieron dos modelos de clasificación vistos en clase, de familias distintas, y se compararon contra una línea base:

- **Regresión logística (candidata a producción):** aprende un peso por cada término de forma conjunta, aprovecha los bigramas (por ejemplo, «no recomiendo»), tiende a entregar probabilidades más confiables que

Naive Bayes y se puede interpretar con sus coeficientes. El desbalance de clases se trata con **class_weight='balanced'**, sin agregar ni eliminar reseñas.
- **Naive Bayes (variante Complement):** modelo probabilístico muy rápido que asume independencia entre términos. Es competitivo con pocos datos y la variante Complement está pensada para clases desbalanceadas.
- **Línea base:** un modelo que siempre predice la clase más frecuente (positivo). Si los modelos no la superan con claridad, no aportan valor.

Se usan dos modelos porque la comparación permite justificar con datos cuál queda en producción, en lugar de asumirlo. Además, al ser de familias distintas (discriminativo frente a generativo), la comparación muestra una decisión de diseño real y no solo dos números.

2.2 Configuración del experimento

- **Datos:** se usan las particiones generadas por **preparar_datos.py** en la Etapa 1: **data/train.csv** (1.433 reseñas) y **data/test.csv** (359 reseñas), con división estratificada, semilla fija (**random_state=42)** y sin textos duplicados. El conjunto de prueba no se usa para ajustar ni elegir nada hasta la evaluación final.
- **Pipeline:** cada modelo se arma como un único **Pipeline** de scikit-learn formado por **crear_preprocesador()** de **preprocesamiento.py** (normalización más TF-IDF) y el clasificador. Así, en cada partición de la validación cruzada el TF-IDF se ajusta solo con los datos de entrenamiento de esa partición (sin fuga de información), y el mismo pipeline completo es el que usará el backend.
- **Validación cruzada: StratifiedKFold** de 5 particiones, con mezcla y semilla fija, aplicada solo sobre el conjunto de entrenamiento.
- **Métricas:** la métrica principal es el **F1 macro**, que promedia el F1 de las tres clases con el mismo peso. Se eligió porque las clases están desbalanceadas (56 % positivas, 28 % negativas y 16 % neutras) y el accuracy por sí solo puede ocultar un mal desempeño en la clase minoritaria. También se reportan el accuracy, el F1 y el recall de la clase neutra, el reporte por clase y la matriz de confusión.
- **Entorno:** Python 3.12.10 y scikit-learn 1.8.0 (versiones fijadas en **requirements.txt**).

2.3 Línea base

El modelo que siempre predice «positivo» obtuvo, en validación cruzada, un accuracy de 0,562 y un F1 macro de 0,240. El accuracy coincide con la proporción de reseñas positivas, y el F1 macro es bajo porque el modelo nunca acierta las clases negativa y neutra. Este es el valor que los modelos deben superar.

2.4 Ajuste de hiperparámetros

Se usó **GridSearchCV** sobre el conjunto de entrenamiento, con validación cruzada estratificada de 5 particiones y F1 macro como métrica de selección. Además de los parámetros del modelo, se probó el rango de n-gramas (unigramas frente a unigramas más bigramas) y la frecuencia mínima de los términos (**min_df)**.

Modelo	Parámetros probados	Mejor combinación	F1 macro (CV)
Regresión logística	n-gramas (1,1) y (1,2); min_df 1, 2 y 3; C 0,1, 1 y 10	C = 1, min_df = 2, n-gramas (1,2)	0,7191
Naive Bayes (Complement)	n-gramas (1,1) y (1,2); min_df 1, 2 y 3; alpha 0,1, 0,5 y 1,0	alpha = 0,1, min_df = 2, n-gramas (1,2)	0,6489

Ambos modelos eligieron unigramas más bigramas y min_df = 2, la misma configuración que usa preprocesamiento.py, lo que confirma la decisión tomada en la Etapa 1. En Naive Bayes, alpha = 0,1 fue el valor más bajo probado, por lo que ampliar la búsqueda hacia valores menores podría mejorar algo su resultado.

```
Regresión logística  (155 s)
  mejores parámetros: {'clf__C': 1, 'prep__tfidf__min_df': 2, 'prep__tfidf__ngram_range': (1, 2)}
  F1 macro (CV): 0.7191

Naive Bayes (Complement)  (130 s)
  mejores parámetros: {'clf__alpha': 0.1, 'prep__tfidf__min_df': 2, 'prep__tfidf__ngram_range': (1, 2)}
  F1 macro (CV): 0.6489
```

2.5 Comparación de los modelos

Resultados de validación cruzada sobre el conjunto de entrenamiento:

Modelo	Accuracy	F1 macro	Desv. F1 macro	F1 neutro	Recall neutro
Línea base	0,5618	0,2398	0,0003	0,0000	0,0000
Regresión logística	0,8025	0,7191	0,0268	0,4403	0,4086
Naive Bayes (Complement)	0,8137	0,6489	0,0401	0,2122	0,1277

- Los dos modelos superan claramente la línea base: el accuracy sube de 0,56 a más de 0,80 y el F1 macro de 0,24 a 0,72 (regresión logística) y 0,65 (Naive Bayes).
- Naive Bayes tiene un accuracy ligeramente mayor, pero reconoce muy pocas reseñas neutras (recall de 0,13 frente a 0,41). El accuracy

se ve bien porque abundan las reseñas positivas y negativas, y por eso se usa el F1 macro como criterio principal.
- La diferencia de F1 macro entre los dos modelos (0,07) es mayor que las desviaciones de la validación cruzada (0,027 y 0,040), por lo que no se explica solo por ruido.

2.6 Modelo elegido para producción

Se eligió la **regresión logística** por las siguientes razones: obtuvo el mayor F1 macro, reconoce mucho mejor la clase neutra, que es la más difícil, entrega probabilidades que el sistema puede mostrar al usuario y permite explicar su decisión mediante los coeficientes de cada término. Naive Bayes queda como alternativa más simple y rápida, pero su mejor accuracy se logra a costa de ignorar casi por completo las reseñas neutras.

```
Modelo: Regresión logística
  Accuracy: 0.7772 | F1 macro: 0.6859
Línea base:
  Accuracy: 0.5627 | F1 macro: 0.2400

              precision    recall  f1-score   support

    negativo      0.810     0.850     0.829       100
      neutro      0.385     0.351     0.367        57
    positivo      0.861     0.861     0.861       202

    accuracy                          0.777       359
   macro avg      0.685     0.687     0.686       359
weighted avg      0.771     0.777     0.774       359
```

2.7 Evaluación final en el conjunto de prueba

El modelo elegido se evaluó una sola vez en data/test.csv (359 reseñas que no se usaron para ajustar nada).

Modelo	Accuracy	F1 macro
Línea base	0,5627	0,2400
Regresión logística	0,7772	0,6859

Resultados por clase:

Clase	Precisión	Recall	F1	Reseñas
Negativo	0,810	0,850	0,829	100
Neutro	0,385	0,351	0,367	57
Positivo	0,861	0,861	0,861	202

El resultado coincide con el de la configuración «sin reducción de palabras, unigramas más bigramas» evaluada en la Etapa 1 (F1 macro de 0,6859 y accuracy de 77,72 %), lo que confirma que se trabajó con la misma división de datos y el mismo preprocesamiento. El F1 macro en prueba (0,686) es algo menor que en validación cruzada (0,719); la prueba contiene solo 57 reseñas neutras, por lo que la estimación de esa clase es más ruidosa.

2.8 Matriz de confusión

Filas: clase real. Columnas: clase predicha.

	Predicho negativo	Predicho neutro	Predicho positivo	Total
Real negativo	85	9	6	100
Real neutro	15	20	22	57
Real positivo	5	23	174	202



El modelo acertó 279 de las 359 reseñas. Casi nunca confunde una reseña claramente positiva con una claramente negativa; sus errores se concentran en la clase neutra.

2.9 Análisis de errores

El modelo cometió 80 errores en 359 reseñas (22,3 %).

Clase real	Errores	Predichas como
Neutro	37 de 57	15 negativo, 22 positivo
Positivo	28 de 202	5 negativo, 23 neutro
Negativo	15 de 100	9 neutro, 6 positivo

- **69 de los 80 errores (86 %) involucran a la clase neutra**: 37 reseñas neutras mal clasificadas y 32 reseñas positivas o negativas predichas como neutras. Solo 11 errores son confusiones directas entre negativo y positivo.
- De las 52 reseñas que el modelo predijo como neutras, solo 20 lo eran (precisión de 0,385).

Posibles explicaciones:

- **Ambigüedad de la clase neutra:** la etiqueta neutra proviene de la puntuación de 3 estrellas, y esas reseñas suelen mezclar aspectos buenos y malos, por lo que su vocabulario se parece al de las otras dos clases. En la revisión manual de la Etapa 1, solo el 80 % de las reseñas neutras coincidía con su etiqueta, frente al 95 % de las positivas y negativas.
- **Efecto del balanceo de clases:** class_weight='balanced' da más peso a la clase neutra, lo que probablemente lleva al modelo a predecirla con frecuencia, a costa de su precisión. Esta explicación es una hipótesis y no se comprobó de forma experimental.

Los ejemplos concretos de reseñas mal clasificadas, con la predicción y la confianza del modelo, se muestran en la sección 8 del notebook notebooks/modelado.ipynb.

2.10 Limitaciones y posibles mejoras

- La clase neutra es la más difícil: F1 de 0,37 en prueba. Se podría equilibrar mejor su precisión y su recall, por ejemplo ajustando los pesos de clase o el umbral de decisión.
- Revisar las etiquetas de 3 estrellas, que son la fuente principal de ambigüedad.
- Usar el título de la reseña como entrada adicional al texto.
- Ampliar la búsqueda de alpha en Naive Bayes hacia valores menores.
- Plantear el problema como estimación de la puntuación de 1 a 5, en lugar de tres clases.
- El corpus corresponde a reseñas de hoteles de Granada (España), por lo que el modelo podría funcionar peor con otros dominios o con español de otras regiones.

2.11 Artefactos y reproducibilidad

Archivo	Contenido
notebooks/modelado.ipynb	Línea base, ajuste, comparación, evaluación final, matriz de confusión, interpretación y análisis de errores
artefactos/modelo.joblib	Pipeline completo (preprocesamiento más regresión logística), listo para modelo.predict([texto])
artefactos/metricas_modelado.json	Métricas de validación cruzada y de prueba, para el endpoint de métricas del backend
artefactos/tabla_comparativa_modelos.csv	Tabla comparativa de los modelos

Para usar el modelo en el backend es necesario que preprocesamiento.py esté disponible en el proyecto, porque el archivo guarda una referencia a la clase TextoNormalizador, y usar la misma versión de scikit-learn (1.8.0). Se agregó matplotlib a requirements.txt, necesario para la matriz de confusión del notebook.

2.12 Declaración de uso de asistentes de IA

Se utilizó un asistente de inteligencia artificial (Claude, de Anthropic) como apoyo para estructurar el notebook de modelado y redactar borradores de esta etapa 2 de la documentación. La ejecución de los experimentos, la interpretación de los resultados y la revisión del contenido fueron realizadas por la integrante responsable, quien puede explicar y modificar todo el código aportado.

ETAPA 3. PREPROCESAMIENTO FINAL Y MODELO EN PRODUCCIÓN

Responsable: Sergio David Cambara Dorbigny, a partir del trabajo de Datos y NLP (Elizabeth Peña Rivero) y de Modelado (Claudia Evelin Tapia Molina)

3.1 Motivación

La Etapa 2 documenta la primera iteración del modelado, hecha sobre el pipeline de la Etapa 1, que normalizaba el texto y aplicaba TF-IDF sin ningún tipo de reducción de palabras. La consigna del proyecto exige implementar stemming y lematización, compararlos con métricas y elegir uno. Por eso se incorporó el stemming al pipeline y se reentrenaron los modelos. Esta etapa documenta el preprocesamiento final, los resultados del reentrenamiento y el modelo que queda en producción.

3.2 Decisiones de preprocesamiento (versión final)

- Minúsculas, sin acentos (se conserva la ñ), sin URLs, números ni signos de puntuación.
- Alargamientos reducidos (`buenooo` pasa a `bueno`).
- Tokenización por palabras; se descartan las de una sola letra.
- Stop words en español, **conservando las negaciones y los intensificadores** (`no`, `sin`, `nada`, `nunca`, `muy`, `poco`, `pero`...), porque cambian el sentido de la reseña.
- **Stemming Snowball en español** (por ejemplo, `recomiendo` pasa a `recom`). Se eligió frente a la lematización y frente a no reducir las palabras porque obtuvo el mejor F1 macro en validación cruzada; la comparación completa está en `resultados_nlp/informe_comparaciones.md`.
- TF-IDF con `sublinear_tf=True`. Por defecto el pipeline usa unigramas y bigramas con `min_df=2`; el modelo final usa los valores elegidos por validación cruzada (unigramas y `min_df=1`).

- La misma función (`src.nlp.preprocesamiento.pasos`) prepara el texto al entrenar, al predecir y en la vista paso a paso del frontend, y una prueba automática lo comprueba. Así no existen dos versiones del preprocesamiento.

3.3 Reentrenamiento y comparación de modelos

Se repitió el proceso de la Etapa 2 (línea base, regresión logística y Naive Bayes Complement, con ajuste por `GridSearchCV` y validación cruzada estratificada de 5 particiones sobre `data/train.csv`), ahora sobre el pipeline con stemming.

Modelo	Accuracy (CV)	F1 macro (CV)	F1 neutro (CV)
Línea base (clase más frecuente)	0,562	0,240	0,000
Regresión logística	0,794	0,720	0,446
Naive Bayes (Complement)	0,812	0,651	0,227

Modelo en producción: regresión logística (`C=1`, unigramas, `min_df=1`, `class_weight='balanced'`, sobre las raíces del stemming). Tiene el mayor F1 macro y reconoce mucho mejor la clase neutra, que es la más difícil (recall de 0,457 frente a 0,137 de Naive Bayes). Naive Bayes tiene algo más de accuracy solo porque acierta más la clase mayoritaria (positivo).

3.4 Evaluación final en el conjunto de prueba

El modelo se evaluó una sola vez en `data/test.csv` (359 reseñas).

Modelo	Accuracy	F1 macro
Línea base	0,563	0,240

Regresión logística con stemming	0,786	0,707

F1 por clase: negativo 0,83, neutro 0,42 y positivo 0,88. El modelo cometió 77 errores en 359 reseñas (21,4 %), y 70 de ellos (91 %) involucran a la clase neutra, igual que en la primera iteración.

3.5 Efecto del stemming

Métrica	Sin stemming (Etapa 2)	Con stemming (Etapa 3)
F1 macro en validación cruzada	0,719	0,720
F1 macro en prueba	0,686	0,707
Accuracy en prueba	0,777	0,786
F1 de la clase neutra en prueba	0,37	0,42
Errores en prueba	80	77

La mejora en validación cruzada es mínima y la mejora en prueba es moderada. Como la prueba contiene solo 57 reseñas neutras, la diferencia en esa clase debe interpretarse con cautela. Aun así, la elección del stemming está respaldada por la comparación del informe de `resultados_nlp/`, y además el pipeline cumple lo que exige la consigna: comparar técnicas de reducción y elegir una con métricas.

3.6 Artefactos y reproducibilidad

- `artefactos/modelo_final.joblib`: pipeline completo (preprocesamiento más modelo), listo para `modelo.predict([texto])`.

- `artefactos/metricas_modelado.json` y `artefactos/tabla_comparativa_modelos.csv`: métricas que consume el backend y que se usan en el informe.
- `notebooks/modelado.ipynb` y el script `src/models/entrenar.py`: con `python -m src.models.entrenar` se pasa del corpus XML a los modelos guardados con un solo comando, con el mismo proceso del notebook.

ETAPA 4. APLICACIÓN WEB: BACKEND, FRONTEND Y DESPLIEGUE

Responsable: Sergio David Cambara Dorbigny (Backend, Frontend e integración)

4.1 Arquitectura

La aplicación tiene dos partes. La **API** carga `artefactos/modelo_final.joblib` y usa `src/nlp/preprocesamiento.py`, por eso se ejecuta desde la raíz del repositorio. El **frontend** llama a la API mediante la ruta `/api`. El backend nunca reimplementa el preprocesamiento: usa el mismo código con el que se entrenó el modelo. Para cargar el archivo `.joblib` hace falta que `preprocesamiento.py` sea importable, porque el archivo guarda una referencia a la clase `TextoNormalizador`, y conviene usar la misma versión de scikit-learn que figura en `artefactos/metadatos.json`.

4.2 Backend (FastAPI)

API REST en `api/main.py`, con los siguientes endpoints:

Método	Ruta	Descripción
GET	`/api/salud`	Estado de la API y del modelo (503 si no hay modelo)
POST	`/api/predecir`	Recibe `{"texto": "..."}` y devuelve sentimiento, confianza, probabilidades y preprocesamiento

POST	`/api/preprocesar`	Recibe `{"texto": "..."}` y devuelve el texto normalizado, los tokens, las stop words eliminadas y las raíces
GET	`/api/metricas`	Métricas de `artefactos/metricas_modelado.json`

- **Validación de entradas:** un texto vacío, con solo signos o números, de más de 5000 caracteres, o un JSON sin el campo `texto`, responde con error 422 y un mensaje claro en español en `detail`. Si el modelo no está cargado, responde 503.
- **Aviso de vocabulario:** si ninguna palabra de la reseña está en el vocabulario, `/api/predecir` no clasifica y devuelve un `aviso`.
- **Documentación:** cada endpoint está documentado en `/docs` (Swagger) y `/redoc` (ReDoc fijado a la versión 2 estable).

4.3 Frontend (React y Tailwind CSS)

Interfaz web en `frontend/`, hecha con **React 19** y **Tailwind CSS 4.1** (plugin `@tailwindcss/vite`, estilos en `frontend/src/styles.css`); necesita Node.js 18 o superior. El diseño sigue el estilo de la plantilla iPortfolio de BootstrapMade: menú lateral oscuro fijo, con botón de hamburguesa en pantallas pequeñas, y una sola página con cuatro secciones:

- **Portada.**
- **Análisis:** el usuario ingresa una reseña y ve la predicción con sus probabilidades, y el preprocesamiento paso a paso.
- **Métricas:** panel con las métricas del modelo y la comparación de los dos modelos.
- **Funcionamiento.**

Desde el menú se enlaza la documentación de la API (`/docs` y `/redoc`), que nginx también redirige al backend.

4.4 Pruebas automatizadas

Se ejecutan con `python -m pytest`:

- `tests/test_api.py`: endpoints, casos de error, documentación y que el preprocesamiento que muestra la API sea el mismo que usa el modelo.

- `tests/test_entrega.py`: verificación de que la entrega de Datos y NLP está completa y es consistente.

4.5 Ejecución y contenedores

Con Docker (recomendado, también para desplegar en la nube):

```shell
docker compose up --build
```

- Aplicación web: `http://localhost:8081`
- API (Swagger): `http://localhost:8002/docs`

Sin Docker, en dos terminales:

```shell
pip install -r api/requirements-docker.txt httpx pytest

uvicorn api.main:app --port 8000
```

```shell
cd frontend

npm ci

npm run dev
```

Los contenedores se definen en `api/Dockerfile`, `frontend/Dockerfile` (con nginx) y `docker-compose.yml`. En la nube, cada contenedor respeta la variable `PORT` de la plataforma y el frontend usa `BACKEND_URL` para saber dónde está la API.